

## QuickSort

```
#include <iostream>
using namespace std;

long Partition(long arr[], long starting_index, long ending_index)
{
    long i = starting_index - 1;
    long R = arr[ending_index];
    for (long j = starting_index; j <= ending_index; j++)
    {
        if (arr[j] <= R)
        {
            i++;
            swap(arr[j], arr[i]);
        }
    }
    return i;
}

void QuickSort(long arr[], long starting_index, long ending_index)
{
    if (ending_index > starting_index)
    {
        long P = Partition(arr, starting_index, ending_index);
        QuickSort(arr, starting_index, P - 1);
        QuickSort(arr, P + 1, ending_index);
    }
}

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
    QuickSort(arr, 0, 8);
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

## MergeSort

```
#include <iostream>
using namespace std;

void Merge(long arr[], long starting_index, long median, long ending_index)
{
    long size1 = median - starting_index + 1;
    long size2 = ending_index - median;

    long* SubArray_1 = new long[size1];
    long* SubArray_2 = new long[size2];

    for (long i = 0; i < size1; i++)
        SubArray_1[i] = arr[starting_index + i];
    for (long j = 0; j < size2; j++)
        SubArray_2[j] = arr[median + 1 + j];

    long i = 0, j = 0, k = starting_index;

    while (i < size1 && j < size2)
    {
        if (SubArray_1[i] <= SubArray_2[j])
        {
            arr[k] = SubArray_1[i];
            i++;
        }
        else
        {
            arr[k] = SubArray_2[j];
            j++;
        }
        k++;
    }
    while (i < size1)
    {
        arr[k] = SubArray_1[i];
        i++;
        k++;
    }
    while (j < size2)
    {
        arr[k] = SubArray_2[j];
        j++;
        k++;
    }
}

void MergeSort(long arr[], long starting_index, long ending_index)
{
    if (starting_index < ending_index)
    {
        long median = starting_index + (ending_index - starting_index) / 2; or m= s + e / 2

        MergeSort(arr, starting_index, median);
        MergeSort(arr, median + 1, ending_index);

        Merge(arr, starting_index, median, ending_index);
    }
}
```

```

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    MergeSort(arr, 0, 8);

    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}

```

## InsertionSort

```

#include <iostream>
using namespace std;

void InsertionSort(long arr[], long size)
{
    for (long i = 1; i < size; i++)
    {
        long key = arr[i];
        long j = i - 1;
        while (arr[j] > key && j >= 0)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    InsertionSort(arr, 9);

    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}

```

## BubbleSort

```
#include <iostream>
using namespace std;

void BubbleSort(long arr[], long size)
{
    for (long i = 0; i < size - 1; i++)
    {
        for (long j = 0; j < size - i - 1; j++)
        {
            if (arr[j + 1] < arr[j])
            {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    BubbleSort(arr, 9);

    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

## SelectionSort

```
#include <iostream>
using namespace std;

void SelectionSort(long arr[], long size)
{
    for (long i = 0; i < size - 1; i++)
    {
        long smallest_index = i; // donot write smallest index
        for (long j = i + 1; j < size; j++)
        {
            // write arr[j] < arr[i]
            if (arr[j] < arr[smallest_index])
            {
                smallest_index = j; // write swap(arr[i],arr[j])
            }
        }
        swap(arr[i], arr[smallest_index]); // already done swapping
    }
}

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    SelectionSort(arr,9);

    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

## HeapSort

```
#include <iostream>
using namespace std;

void Heapify(long arr[], long n, long i)
{
    long largest = i;
    long LChild = 2 * i + 1;
    long RChild = 2 * i + 2;

    if (LChild < n && arr[LChild] > arr[largest])
        largest = LChild;

    if (RChild < n && arr[RChild] > arr[largest])
        largest = RChild;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        Heapify(arr, n, largest);
    }
}

void HeapSort(long arr[], long n)
{
    for (long i = n / 2 - 1; i >= 0; i--)
        Heapify(arr, n, i);

    for (long i = n - 1; i >= 0; i--)
    {
        swap(arr[0], arr[i]);
        Heapify(arr, i, 0);
    }
}

void main()
{
    long arr[9] = { 0 };
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    for (long i = 0; i < 9; i++)
        cin >> arr[i];
    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;

    HeapSort(arr, 9);

    for (long i = 0; i < 9; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```